

Predicting Pavement Roughness (IRI) from Traffic and Climate Using Regression

Complete Code Guide: every cell of my notebook, explained, with its real output

LTPP pavement performance data | Python, pandas, NumPy, scikit-learn | VS Code and Jupyter

David Badu

September 2026

Contents

How to use this guide

Step 0: Setting up and preparing the data

Step 1: Loading and inspecting the data

Step 2: Understanding the target (IRI)

Step 3: Exploring the predictors

Step 4: Preparing the features

Step 5: Splitting into training and test sets

Step 6: Linear regression from scratch

Step 7: Evaluating the model

Step 8: Polynomial features and regularisation

Step 9: Adding previous IRI

Conclusions

Summary of results

How to use this guide

This guide is built directly from my finished notebook, *Predicting Pavement Roughness (IRI) from Traffic and Climate Using Regression.ipynb*, after it was run from top to bottom (cells numbered 1 to 86, no errors). It follows the notebook cell by cell. For each code cell you will find:

- **An explanation** of what the code does and why, in plain words, before the code.
- **The code itself**, exactly as it is in the notebook (grey box).
- **The real output** it produced (blue box) and any chart, followed by a green box explaining what the output shows.
- My own Markdown notes from the notebook appear in orange boxes.

Long outputs, such as whole tables, are shortened; the notebook holds the full versions.

Reading tips

Code in the text looks like `this`. A name followed by brackets, such as `df.head()`, is a function or method: a command that does something.

Lines starting with `#` inside code are comments, ignored by Python.

Only the **last line** of a cell is displayed automatically; use `print()` or a separate cell for anything else.

Run cells in order. After restarting VS Code, use **Restart** then **Run All**.

The project in one paragraph

The International Roughness Index (IRI) measures how rough a road surface is, in m/km (lower is smoother). Using the US Long-Term Pavement Performance (LTPP) data, the project asks how well traffic, climate and section history predict IRI, and which factors matter most. It applies every major idea from the *Supervised Machine Learning: Regression and Classification* course (linear regression, gradient descent, feature scaling, polynomial features, regularisation, the bias and variance trade-off, logistic regression) and goes beyond it with decision trees, random forests and checks for data leakage.

Map of the steps

Step	What is done	Main idea
0	Set up VS Code and prepare the data	Environment, merging, cleaning
1	Load and inspect	pandas basics, panel data
2	Study the target (IRI)	Histogram, skewness, log, condition classes
3	Explore the predictors	Scatter, groupby, box plots, correlation
4	Prepare the features	X and y, dummy variables, scaling
5	Train/test split	Grouped split, leakage
6	Linear regression from scratch	Cost, gradient, gradient descent, learning rate
7	Evaluate	MAE, RMSE, R^2 , baselines, residuals
8	Polynomials and regularisation	Overfitting, ridge, cross-validation
9	Previous IRI	Lagged features, persistence
10	Trees and forests	Site leakage, feature importance
11	Classification	Logistic regression, precision, recall, thresholds

Step 0: Setting up and preparing the data

0.1 The working environment

The notebook runs in **VS Code** with **Jupyter**, inside a virtual environment (**.venv**) using Python 3.14. The packages are installed once from the VS Code terminal:

Terminal

```
pip install pandas numpy matplotlib scikit-learn ipykernel
```

- **pandas**: tables of data.
- **numpy**: fast maths on arrays.
- **matplotlib**: charts.
- **scikit-learn**: ready-made models and tools.
- **ipykernel**: lets VS Code run the notebook with **.venv**.

0.2 Where the data came from

File	Sheets used	Contents
Bucket_146230.xlsx	ANALYSIS_IRI	IRI per section and visit
	TRF_TREND (x3)	Annual ESAL, trucks per day (AADTT), trucks by vehicle class
	EXPERIMENT_SECTION	Construction events, pavement family, GPS or SPS
	SHRP_INFO	Functional class
Bucket_146231.xlsx	MERRA_TEMP_YEAR	Annual temperature, freezing index, freeze-thaw cycles
	LTPP_Section_Reference	Link from climate grid cell to section

A merge script averaged the repeated IRI runs of each visit, took the year from the visit date, and joined traffic and climate to each section and year, giving **pavement_merged.csv** (15,985 rows, 29 columns).

0.3 The cleaning script

clean_data.py turns the merged file into **pavement_clean.csv**, the file the notebook reads.

clean_data.py

```
"""
Step 4: clean pavement_merged.csv and add section attributes (age, pavement family,
functional class) from Bucket_146230.xlsx.

Run from the project folder: python clean_data.py
Output: pavement_clean.csv
"""
import numpy as np
import pandas as pd

MERGED = "pavement_merged.csv"
IMS = "Bucket_146230.xlsx"
OUT = "pavement_clean.csv"
KEY = ["STATE_CODE", "SHRP_ID", "CONSTRUCTION_NO"]

d = pd.read_csv(MERGED, dtype={"SHRP_ID": str})
```

```

log = [f"Rows in merged file: {len(d)}"]

# 1. Exact duplicate rows
n = len(d); d = d.drop_duplicates()
log.append(f"Dropped exact duplicates: {n - len(d)}")

# 2. Section ID. SHRP_ID repeats across states, so a section is STATE_CODE + SHRP_ID.
d["SECTION_ID"] = d.STATE_CODE.astype(str).str.zfill(2) + "_" + d.SHRP_ID

# 3. Fill the few missing state names from other rows with the same STATE_CODE
names = d.dropna(subset=["STATE_CODE_EXP"]).groupby("STATE_CODE").STATE_CODE_EXP.first()
d["STATE_CODE_EXP"] = d.STATE_CODE_EXP.fillna(d.STATE_CODE.map(names))

# 4. Join construction event data (date each CONSTRUCTION_NO began, pavement family,
experiment)
es = pd.read_excel(IMS, sheet_name="EXPERIMENT_SECTION", dtype={"SHRP_ID": str})
es = es[KEY + ["CN_ASSIGN_DATE", "CN_CHANGE_REASON_EXP", "GPS_SPS",
               "EXPERIMENT_NO_EXP", "PAVEMENT_FAMILY_EXP"]]
d = d.merge(es, on=KEY, how="left", validate="many_to_one")

# 5. Functional class (one per section; take the most recent SHRP_INFO record)
si = pd.read_excel(IMS, sheet_name="SHRP_INFO", dtype={"SHRP_ID": str})
si = (si.sort_values("START_DATE").groupby(["STATE_CODE", "SHRP_ID"]).last()
      [{"FUNC_CLASS_EXP"}]).reset_index()
d = d.merge(si, on=["STATE_CODE", "SHRP_ID"], how="left", validate="many_to_one")

# 6. Engineered features
d["AGE_SINCE_CN"] = d.YEAR - d.CN_ASSIGN_DATE.dt.year          # years since this
construction event
d["IS_REHAB"] = (d.CONSTRUCTION_NO > 1).astype(int)            # 1 = section has been
rehabilitated
d["PAVEMENT_TYPE"] = np.where(d.PAVEMENT_FAMILY_EXP.str.startswith("Asphalt"),
                              "Flexible", "Rigid")
d["URBAN"] = d.FUNC_CLASS_EXP.str.startswith("Urban").astype("Int64")
d["SHARE_CLASS9"] = d.AADTT_VEH_CLASS_9_TREND / d.AADTT_ALL_TRUCKS_TREND.replace(0,
np.nan)
d["LOG_ESAL"] = np.log1p(d.ANNUAL_ESAL_TREND)
d["LOG_AADTT"] = np.log1p(d.AADTT_ALL_TRUCKS_TREND)

# 7. Drop columns that are redundant, too sparse, or would leak the target
drop = {
    "ANNUAL_TRUCK_VOLUME_TREND": "= AADTT x days counted; redundant and 11% missing",
    "CMLTV_VOL_VEH_CLASS_9_TREND": "24% missing; cumulative, depends on start of count",
    "TEMP_MEAN_AVG": "r = 1.00 with TEMP_AVG",
    "ANNUAL_GESAL_TREND": "r = 0.95 with ANNUAL_ESAL_TREND",
    "IRI_LEFT_WHEEL_PATH": "target leakage (part of IRI_AVG)",
    "IRI_RIGHT_WHEEL_PATH": "target leakage (part of IRI_AVG)",
    "MRI": "target leakage (identical to IRI_AVG)",
}
d = d.drop(columns=list(drop))
log += [f"Dropped {c}: {why}" for c, why in drop.items()]

# 8. Rows with no traffic data
n = len(d); d = d.dropna(subset=["ANNUAL_ESAL_TREND", "AADTT_VEH_CLASS_9_TREND"])
log.append(f"Dropped rows missing traffic: {n - len(d)}")

```

```
# 9. Sanity checks
assert d.IRI_AVG.between(0.3, 6).all(), "IRI outside plausible range"
assert (d.AGE_SINCE_CN >= 0).all(), "Survey year before construction date"

d = d.sort_values(["SECTION_ID", "CONSTRUCTION_NO", "YEAR"])
d.to_csv(OUT, index=False)
log += [f"Rows out: {len(d)}", f"Sections: {d.SECTION_ID.nunique()}",
        f"Columns: {d.shape[1]}", f"Years: {d.YEAR.min()}-{d.YEAR.max()}"]
print("\n".join(log))
print("\nMissing values remaining:\n", d.isna().sum()[d.isna().sum() > 0].to_string())
```

Output

```
Rows in merged file: 15985
Dropped exact duplicates: 35
Dropped ANNUAL_TRUCK_VOLUME_TREND: = AADTT x days counted; redundant and 11% missing
Dropped CMLTV_VOL_VEH_CLASS_9_TREND: 24% missing; cumulative, depends on start of count
Dropped TEMP_MEAN_AVG: r = 1.00 with TEMP_AVG
Dropped ANNUAL_GESAL_TREND: r = 0.95 with ANNUAL_ESAL_TREND
Dropped IRI_LEFT_WHEEL_PATH: target leakage (part of IRI_AVG)
Dropped IRI_RIGHT_WHEEL_PATH: target leakage (part of IRI_AVG)
Dropped MRI: target leakage (identical to IRI_AVG)
Dropped rows missing traffic: 10
Rows out: 15940
Sections: 1550
Columns: 36
Years: 1989-2025
```

1. **Duplicates:** 35 exact copies removed.
2. **Section ID:** SHRP_ID repeats across states, so a section is state code plus SHRP_ID (1,550 sections, not 671).
3. **Joins:** construction dates, pavement family and experiment (EXPERIMENT_SECTION); functional class (SHRP_INFO).
4. **New features:** AGE_SINCE_CN, IS_REHAB, PAVEMENT_TYPE, URBAN, SHARE_CLASS9, LOG_ESAL, LOG_AADTT.
5. **Dropped columns:** wheel-path IRI and MRI are the target in another form (**target leakage**); others were duplicates or too sparse.
6. **Checks:** **assert** stops the script if IRI is implausible or a survey predates its construction.

Limitation to remember

For first construction events, AGE_SINCE_CN counts from entry into LTPP, not from original construction (true dates are in LTPP table INV_AGE).

Step 1: Loading and inspecting the data

Every notebook starts by loading its tools. `import pandas as pd` loads pandas (tables) with the short name `pd`; `numpy (np)` does maths on arrays; `matplotlib.pyplot (plt)` draws charts. `pd.read_csv` reads the file into a **DataFrame** called `df`, a table with named columns. `dtype={"SHRP_ID": str}` keeps IDs like "0502" as text so the leading zero is not lost. The file name has no folder in front of it, so the notebook works on any computer as long as the CSV sits in the same folder. `print(df)` shows the first and last rows.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("pavement_clean.csv", dtype={"SHRP_ID": str})

print(df)
```

Output

	STATE_CODE	SHRP_ID	CONSTRUCTION_NO	YEAR	N_RUNS	IRI_AVG	\
0	1	0502	1	1991	5	1.0360	
1	1	0502	2	1992	4	0.7745	
2	1	0502	2	1994	5	0.8540	
3	1	0502	2	1996	5	0.8193	
4	1	0502	2	1999	5	0.8143	
...	...	...	...	...	...	...	
15935	90	B351	2	1994	5	1.6446	
15936	90	B351	2	1995	10	1.9259	
15937	90	B351	3	1997	5	2.4953	
15938	90	B351	3	1998	5	2.5331	
15939	90	B351	3	1999	5	2.7157	
	STATE_CODE_EXP	ANNUAL_ESAL_TREND	AADTT_ALL_TRUCKS_TREND	\			
0	Alabama	76486.0	402.0				
1	Alabama	78986.0	415.0				
2	Alabama	54000.0	241.0				
3	Alabama	89030.0	467.0				
4	Alabama	96276.0	510.0				
...	...	...	...				
15935	Saskatchewan	113836.0	169.0				
15936	Saskatchewan	90000.0	181.0				
15937	Saskatchewan	130000.0	212.0				
15938	Saskatchewan	92889.0	185.0				
15939	Saskatchewan	97791.0	180.0				
	AADTT_VEH_CLASS_4_TREND	...	EXPERIMENT_NO_EXP	\			
0	0.0	...	AC Overlay of AC Pavement				
1	0.0	...	AC Overlay of AC Pavement				
2	1.0	...	AC Overlay of AC Pavement				
... (45 more lines not shown)							

What the output shows

The table has **15,940 rows and 36 columns**. Each row is one road section measured in one year.

`df.info()` lists every column with its data type and how many values are present.

`df["IRI_AVG"].describe()` summarises the target, and `df["SECTION_ID"].nunique()` counts distinct sections. Only the last line of a cell is shown automatically, which is why `info()` (which prints itself) and the last line appear but the middle one does not.

Code

```
df.info()
df["IRI_AVG"].describe()
df["SECTION_ID"].nunique()
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 15940 entries, 0 to 15939
Data columns (total 36 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   STATE_CODE                            15940 non-null  int64
1   SHRP_ID                              15940 non-null  str
2   CONSTRUCTION_NO                       15940 non-null  int64
3   YEAR                                  15940 non-null  int64
4   N_RUNS                                15940 non-null  int64
5   IRI_AVG                               15940 non-null  float64
6   STATE_CODE_EXP                        15940 non-null  str
7   ANNUAL_ESAL_TREND                    15940 non-null  float64
8   AADTT_ALL_TRUCKS_TREND                15940 non-null  float64
9   AADTT_VEH_CLASS_4_TREND               15940 non-null  float64
10  AADTT_VEH_CLASS_5_TREND               15940 non-null  float64
11  AADTT_VEH_CLASS_6_TREND               15940 non-null  float64
12  AADTT_VEH_CLASS_7_TREND               15940 non-null  float64
13  AADTT_VEH_CLASS_8_TREND               15940 non-null  float64
14  AADTT_VEH_CLASS_9_TREND               15940 non-null  float64
15  AADTT_VEH_CLASS_10_TREND              15940 non-null  float64
16  AADTT_VEH_CLASS_11_TREND              15940 non-null  float64
17  AADTT_VEH_CLASS_12_TREND              15940 non-null  float64
18  AADTT_VEH_CLASS_13_TREND              15940 non-null  float64
19  TEMP_AVG                              15940 non-null  float64
20  FREEZE_INDEX                          15940 non-null  int64
21  FREEZE_THAW                           15940 non-null  int64
22  SECTION_ID                            15940 non-null  str
... (16 more lines not shown)
```

What the output shows

No column has missing values, and there are **1,550 sections**.

`df.describe()` gives count, mean, standard deviation, minimum, quartiles and maximum for every numeric column. `print()` forces the result to display.

Code

```
print(df.describe())
```

Output

	STATE_CODE	CONSTRUCTION_NO	YEAR	N_RUNS	\
count	15940.000000	15940.000000	15940.000000	15940.000000	

```

mean      33.436763      2.390402      1998.925282      5.430552
std       22.849748      1.588858      7.137056      2.220288
min        1.000000      1.000000      1989.000000      1.000000
25%       17.000000      1.000000      1993.000000      5.000000
50%       30.000000      2.000000      1998.000000      5.000000
75%       48.000000      3.000000      2003.000000      5.000000
max       90.000000     15.000000     2025.000000     45.000000

      IRI_AVG  ANNUAL_ESAL_TREND  AADTT_ALL_TRUCKS_TREND  \
count  15940.000000      1.594000e+04      15940.000000
mean     1.346284      3.400738e+05      920.566562
std      0.615442      4.888482e+05      1066.995860
min      0.319400      0.000000e+00      1.000000
25%      0.929175      5.346000e+04      260.000000
50%      1.196550      1.469820e+05      552.000000
75%      1.587225      4.000000e+05      1204.000000
max      5.742900      4.258367e+06     15170.000000

      AADTT_VEH_CLASS_4_TREND  AADTT_VEH_CLASS_5_TREND  \
count      15940.000000      15940.000000
mean        20.542284        154.967817
std         31.031042        177.469479
min          0.000000         1.000000
25%          4.000000        52.000000
50%         11.000000       113.000000
75%         23.000000       199.000000
max         798.000000      5319.000000

      AADTT_VEH_CLASS_6_TREND  ...  AADTT_VEH_CLASS_13_TREND  TEMP_AVG  \
... (27 more lines not shown)

```

What the output shows

For IRI: minimum **0.319**, median (the 50% row) **1.197**, maximum **5.743 m/km**. The mean (1.346) is above the median, an early sign that a few very rough roads pull the average up.

`df.groupby("SECTION_ID").size()` counts the rows for each section. The second line filters the table to one section, **01_0502**, and shows its history. Only this second result is displayed, because it is the last line.

Code

```

df.groupby("SECTION_ID").size().describe()
df[df["SECTION_ID"] == "01_0502"][["YEAR", "CONSTRUCTION_NO", "IRI_AVG"]]

```

Output

	YEAR	CONSTRUCTION_NO	IRI_AVG
0	1991	1	1.0360
1	1992	2	0.7745
2	1994	2	0.8540
3	1996	2	0.8193
4	1999	2	0.8143
5	2001	2	0.8377
6	2002	2	0.8422
7	2003	2	0.8877
8	2004	2	0.9545
9	2005	2	1.0105
10	2006	2	1.0141
11	2008	2	1.1096
12	2009	2	1.0689
13	2013	2	1.2232
14	2014	2	1.3601
15	2015	2	1.2777
16	2017	2	1.2893

What the output shows

Section 01_0502 was measured 17 times between 1991 and 2017. Repeated measurements of the same units over time are called **panel data**. They are not independent, which is why later splits keep each road on one side.

Your note in the notebook

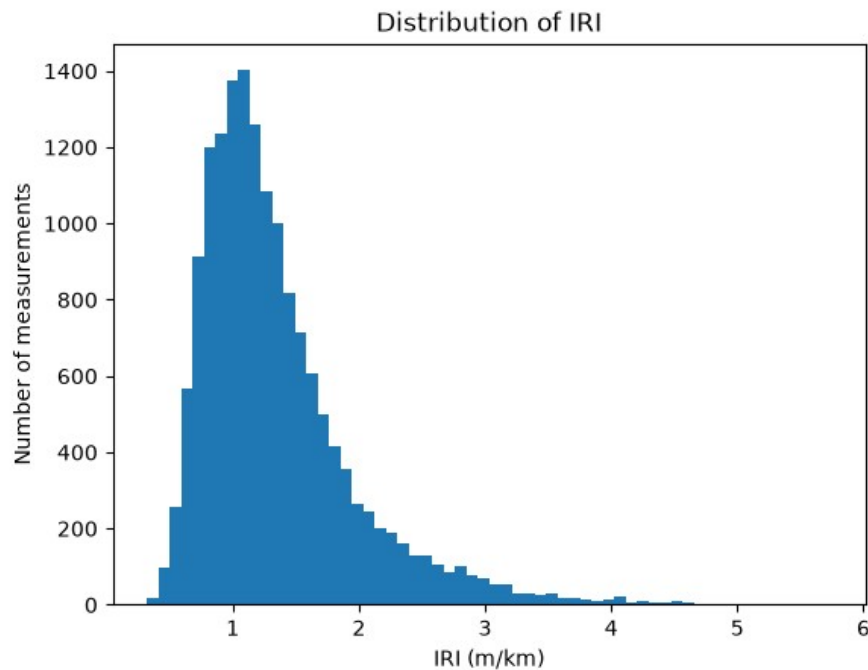
Note: 15,940 rows but only 1,550 sections: each section was surveyed about 10 times (panel data). Surveys of the same road are not independent, so train/test splits must keep each road on one side.

Step 2: Understanding the target (IRI)

A **histogram** sorts values into `bins=60` ranges and draws one bar per range. The label lines only add text, and `plt.show()` displays the chart.

Code

```
plt.hist(df["IRI_AVG"], bins=60)
plt.xlabel("IRI (m/km)")
plt.ylabel("Number of measurements")
plt.title("Distribution of IRI")
plt.show()
```



What the output shows

Most roads sit between about 0.7 and 1.6 m/km, with a long tail of rough roads to the right.

`.skew()` measures how lopsided the distribution is: about 0 is symmetric, positive means a long right tail, above about 1 is strong.

Code

```
print(df["IRI_AVG"].skew())
```

Output

```
1.7100164230236399
```

What the output shows

Skewness is **1.71**: strongly right-skewed.

`np.log` takes the natural logarithm of every value, and `df["LOG_IRI"] = ...` stores it as a new column. A log squeezes large numbers more than small ones, which pulls in a long tail.

Code

```
df["LOG_IRI"] = np.log(df["IRI_AVG"])
print(df["LOG_IRI"].skew())
```

Output

```
0.35096673599308564
```

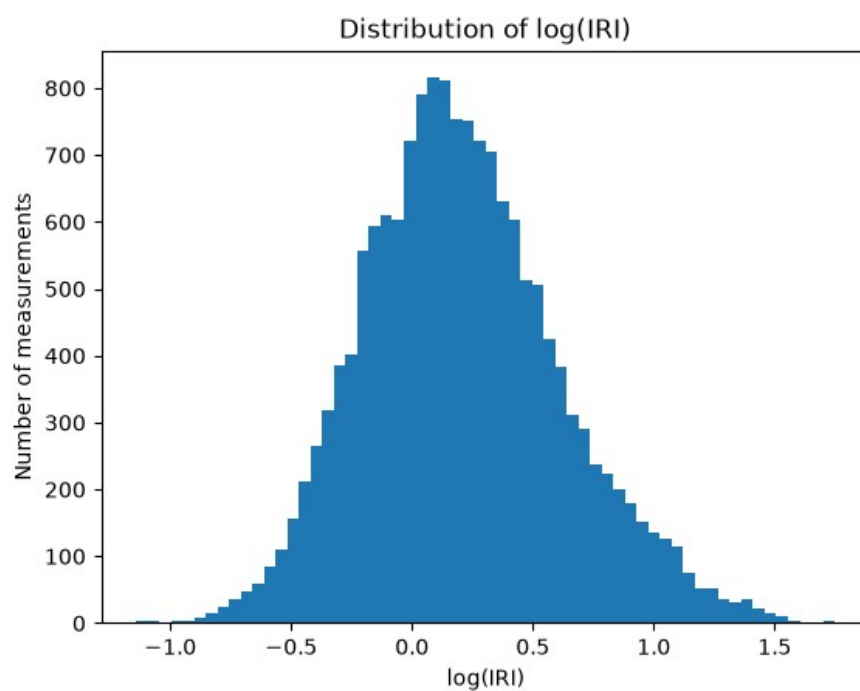
What the output shows

Skewness falls to 0.35, nearly symmetric.

The same histogram, now for **LOG_IRI**.

Code

```
plt.hist(df["LOG_IRI"], bins=60)
plt.xlabel("log(IRI)")
plt.ylabel("Number of measurements")
plt.title("Distribution of log(IRI)")
plt.show()
```

**What the output shows**

The log version looks much more like a bell curve. It was later tested as a target in Step 7.

Your note in the notebook

IRI is right-skewed (1.71). The log transform reduces this to 0.35, so log(IRI) will be tested as the target.

Pavement condition classes (FHWA: Good < 1.50, Fair 1.50 to 2.68, Poor > 2.68 m/km)

`pd.cut` sorts each IRI into FHWA bands. The `bins` list gives the edges (0, 1.50, 2.68, infinity) and `labels` names the bands. `np.inf` means no upper limit. `.value_counts()` counts each band.

Code

```
df["CONDITION"] = pd.cut(df["IRI_AVG"],
                          bins=[0, 1.50, 2.68, np.inf],
                          labels=["Good", "Fair", "Poor"])
df["CONDITION"].value_counts()
```

Output

```
CONDITION
Good      11279
Fair       3966
Poor        695
Name: count, dtype: int64
```

`normalize=True` turns counts into fractions (0.708 means 70.8%).

Code

```
df["CONDITION"].value_counts(normalize=True)
```

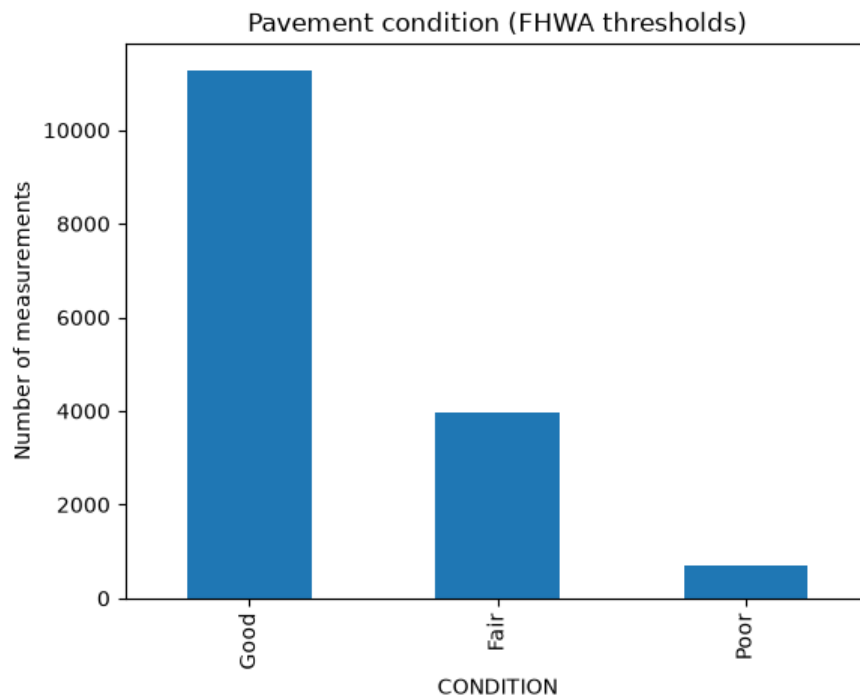
Output

```
CONDITION
Good      0.707591
Fair      0.248808
Poor      0.043601
Name: proportion, dtype: float64
```

`.plot(kind="bar")` draws a bar chart directly from the counts.

Code

```
df["CONDITION"].value_counts().plot(kind="bar")
plt.ylabel("Number of measurements")
plt.title("Pavement condition (FHWA thresholds)")
plt.show()
```

**Your note in the notebook**

Note: 70.8% Good, 24.9% Fair, 4.4% Poor. The classes are imbalanced: a model that always says "Good" is 70.8% accurate but finds no poor roads, so accuracy alone is misleading.

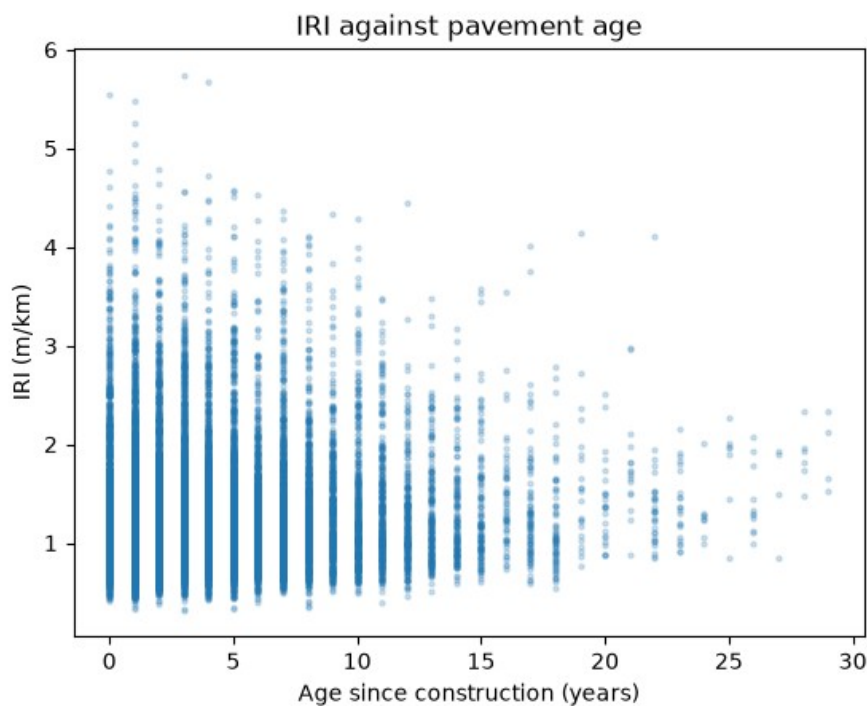
Step 3: Exploring the predictors

3a. IRI and pavement age

`plt.scatter(x, y)` draws one dot per row. `s=5` makes dots small and `alpha=0.2` makes them see-through, so crowded areas look darker. The vertical stripes appear because age is a whole number.

Code

```
plt.scatter(df["AGE_SINCE_CN"], df["IRI_AVG"], s=5, alpha=0.2)
plt.xlabel("Age since construction (years)")
plt.ylabel("IRI (m/km)")
plt.title("IRI against pavement age")
plt.show()
```



`groupby("AGE_SINCE_CN")["IRI_AVG"].median()` splits the rows by age and finds the median IRI of each group. The median is used because IRI is skewed and the median is not pulled up by a few very rough roads.

Code

```
median_by_age = df.groupby("AGE_SINCE_CN")["IRI_AVG"].median()
median_by_age
```

Output

```
AGE_SINCE_CN
0    1.199800
1    1.184400
2    1.171900
3    1.209400
4    1.182200
5    1.222400
6    1.180050
7    1.247000
8    1.213250
```

```

9      1.231500
10     1.178150
11     1.190625
12     1.129500
13     1.162500
14     1.173400
15     1.175400
16     1.350400
17     1.202400
18     1.046700
19     1.474850
20     1.360500
21     1.702032
22     1.319300
23     1.344150
24     1.250350
25     1.955000
26     1.300500
... (4 more lines not shown)

```

What the output shows

From age 0 to 15 the median stays between about 1.13 and 1.25 m/km. Beyond that it jumps around.

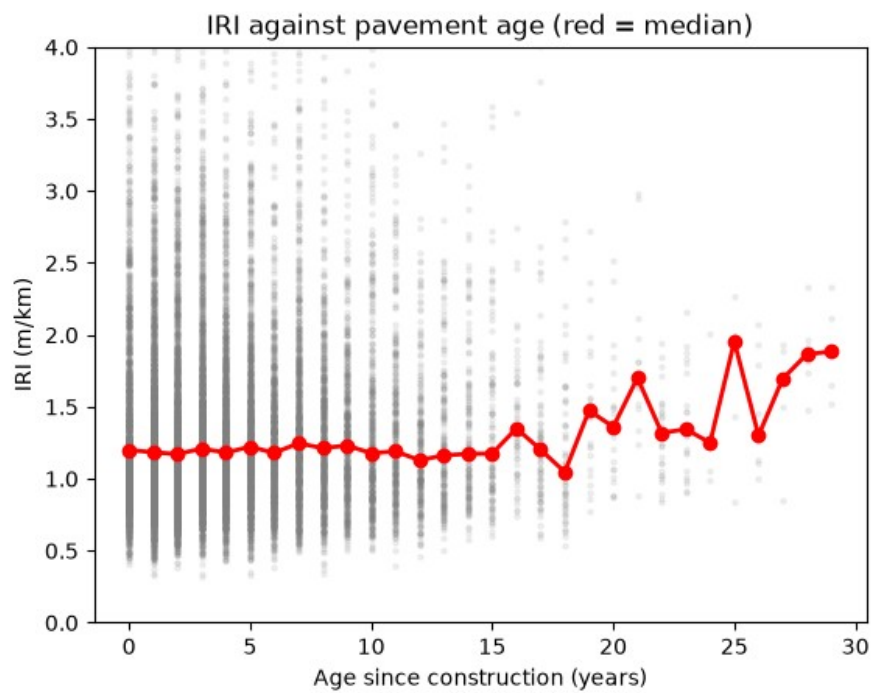
The scatter again in grey, with the medians drawn as a red line using `plt.plot` (which joins points). `plt.ylim(0, 4)` only zooms the view.

Code

```

plt.scatter(df["AGE_SINCE_CN"], df["IRI_AVG"], s=5, alpha=0.1, color="grey")
plt.plot(median_by_age.index, median_by_age.values, color="red", marker="o",
linewidth=2)
plt.xlabel("Age since construction (years)")
plt.ylabel("IRI (m/km)")
plt.title("IRI against pavement age (red = median)")
plt.ylim(0, 4)
plt.show()

```



`.value_counts().sort_index()` counts the measurements at each age, in age order.

Code

```
df["AGE_SINCE_CN"].value_counts().sort_index()
```

Output

```
AGE_SINCE_CN
0      1552
1      2347
2      2058
3      2087
4      1607
5      1319
6       966
7       940
8       704
9       507
10      438
11      343
12      256
13      193
14      145
15      116
16       56
17       84
18       67
19       22
20       24
21       16
22       31
23       22
24        8
25        7
26       11
```

```
... (4 more lines not shown)
```

What the output shows

Over 2,000 measurements at age 1, but only 4 at age 29.

`df["AGE_SINCE_CN"] > 20` gives True or False for each row, and `.sum()` counts the Trues (True counts as 1).

Code

```
(df["AGE_SINCE_CN"] > 20).sum()
```

Output

```
np.int64(109)
```

What the output shows

Only **109** measurements are above age 20. Small samples give unstable medians, which is why the red line jumps at older ages.

To remove differences between roads, we measure how much each road changed since its own first survey. `sort_values` puts surveys in time order. `groupby(...).transform("first")` finds each road's first IRI (per construction event) and copies it onto every row of that road, so it can be subtracted row by row.

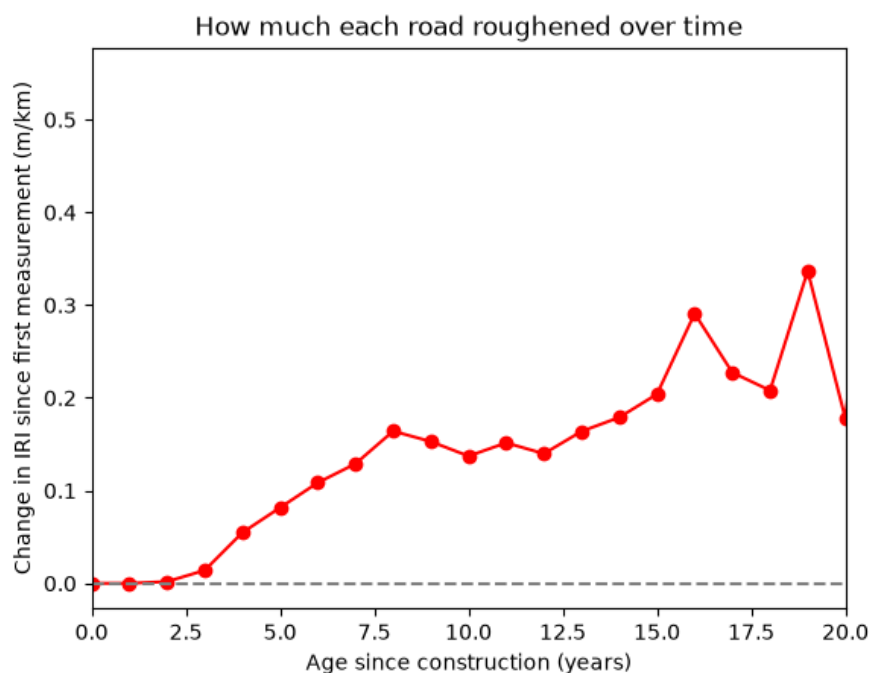
Code

```
df = df.sort_values(["SECTION_ID", "CONSTRUCTION_NO", "YEAR"])
first_iri = df.groupby(["SECTION_ID", "CONSTRUCTION_NO"])["IRI_AVG"].transform("first")
df["IRI_CHANGE"] = df["IRI_AVG"] - first_iri
```

The median change is plotted against age, with a dashed reference line at zero (`plt.axhline`).

Code

```
change_by_age = df.groupby("AGE_SINCE_CN")["IRI_CHANGE"].median()
plt.plot(change_by_age.index, change_by_age.values, marker="o", color="red")
plt.axhline(0, color="grey", linestyle="--")
plt.xlabel("Age since construction (years)")
plt.ylabel("Change in IRI since first measurement (m/km)")
plt.title("How much each road roughened over time")
plt.xlim(0, 20)
plt.show()
```



What the output shows

Within each road, IRI clearly rises, by about 0.2 m/km over 15 years, even though the raw medians looked flat.

Your note in the notebook

Note: median IRI is almost flat with age across all roads (1.13 to 1.25 m/km for ages 0 to 15), but each road's own IRI rises by about 0.2 m/km over 15 years. Rehabilitation resets age (survivorship bias) and roads start at different roughness, which hides the trend.

3b. IRI by pavement type

Your note in the notebook

Rigid pavements are rougher: median 1.84 vs 1.17 m/km for flexible.

The same `groupby` pattern, now splitting by pavement type.

Code

```
df.groupby("PAVEMENT_TYPE")["IRI_AVG"].median()
```

Output

```
PAVEMENT_TYPE
Flexible    1.173000
Rigid      1.842475
Name: IRI_AVG, dtype: float64
```

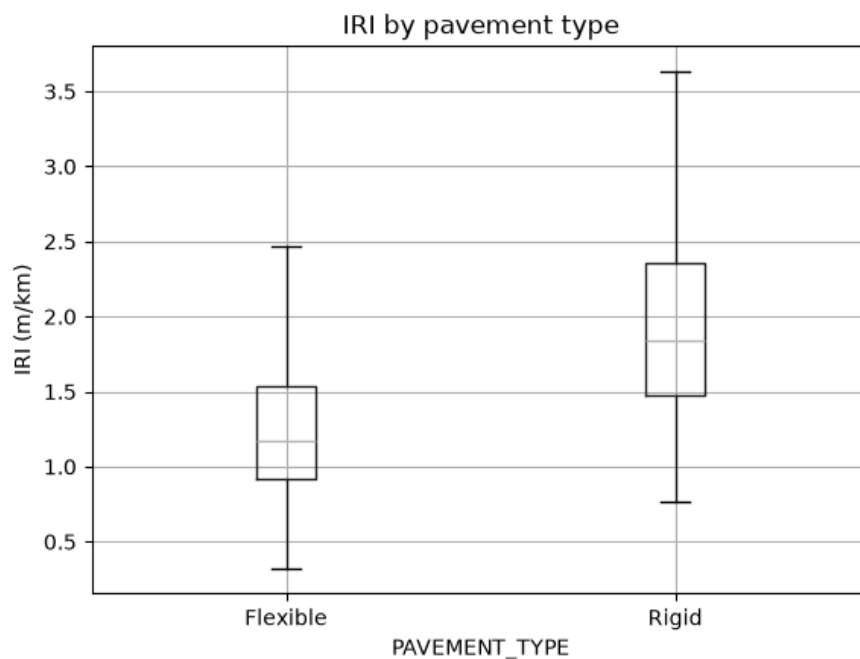
What the output shows

Rigid (concrete) median **1.84** against flexible (asphalt) **1.17 m/km**.

A **box plot**: the line in each box is the median, the box covers the middle 50% of values, and the whiskers show the typical range. `showfliers=False` hides extreme points; `plt.suptitle("")` removes an automatic extra title.

Code

```
df.boxplot(column="IRI_AVG", by="PAVEMENT_TYPE", showfliers=False)
plt.ylabel("IRI (m/km)")
plt.title("IRI by pavement type")
plt.suptitle("")
plt.show()
```



3c. IRI and traffic

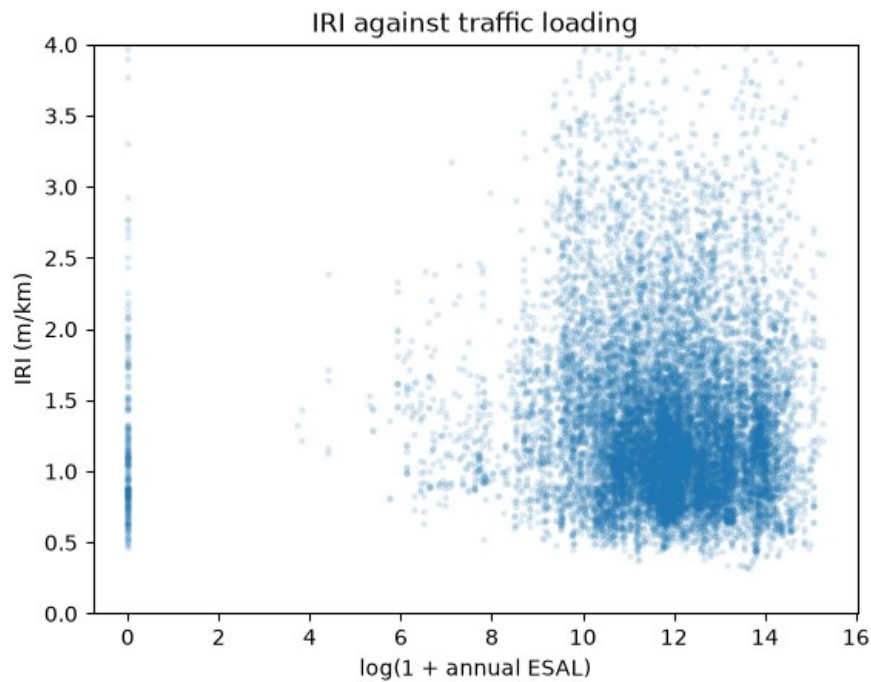
Your note in the notebook

No clear upward trend. Busier roads are built thicker and maintained more (confounding).

`LOG_ESAL` is $\log(1 + \text{annual ESAL})$. Traffic is very skewed, and the log spreads the dots out; the `1 +` avoids $\log(0)$.

Code

```
plt.scatter(df["LOG_ESAL"], df["IRI_AVG"], s=5, alpha=0.1)
plt.xlabel("log(1 + annual ESAL)")
plt.ylabel("IRI (m/km)")
plt.title("IRI against traffic loading")
plt.ylim(0, 4)
plt.show()
```



3d. IRI and climate

`plt.subplots(1, 3)` makes one figure with three charts, stored in `axes[0]`, `axes[1]` and `axes[2]`. `for i, col in enumerate(climate):` repeats the indented lines once per climate column, with `i` counting 0, 1, 2. With subplots, each chart is labelled with `axes[i].set_xlabel(...)`.

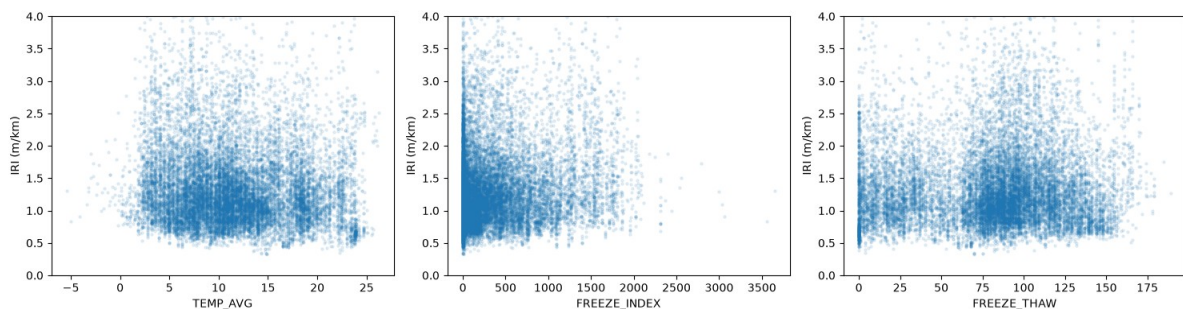
Code

```
climate = ["TEMP_AVG", "FREEZE_INDEX", "FREEZE_THAW"]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for i, col in enumerate(climate):
    axes[i].scatter(df[col], df["IRI_AVG"], s=5, alpha=0.1)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("IRI (m/km)")
    axes[i].set_ylim(0, 4)

plt.tight_layout()
plt.show()
```



3e. Correlation

`.corr()` computes the **correlation coefficient r** between every pair of columns (+1 perfect rise together, 0 none, -1 perfect opposite). `["IRI_AVG"]` keeps each feature's correlation with IRI and `.drop("IRI_AVG")` removes IRI's correlation with itself.

Code

```
features = ["AGE_SINCE_CN", "IS_REHAB", "LOG_ESAL", "LOG_AADTT", "SHARE_CLASS9",
            "TEMP_AVG", "FREEZE_INDEX", "FREEZE_THAW", "URBAN"]

corr_with_iri = df[features + ["IRI_AVG"]].corr()["IRI_AVG"].drop("IRI_AVG")
corr_with_iri.sort_values()
```

Output

```
LOG_AADTT      -0.108473
TEMP_AVG       -0.091429
IS_REHAB       -0.085153
SHARE_CLASS9   -0.058005
LOG_ESAL       -0.027503
AGE_SINCE_CN   -0.001216
URBAN          0.035866
FREEZE_THAW    0.038113
FREEZE_INDEX   0.121047
Name: IRI_AVG, dtype: float64
```

What the output shows

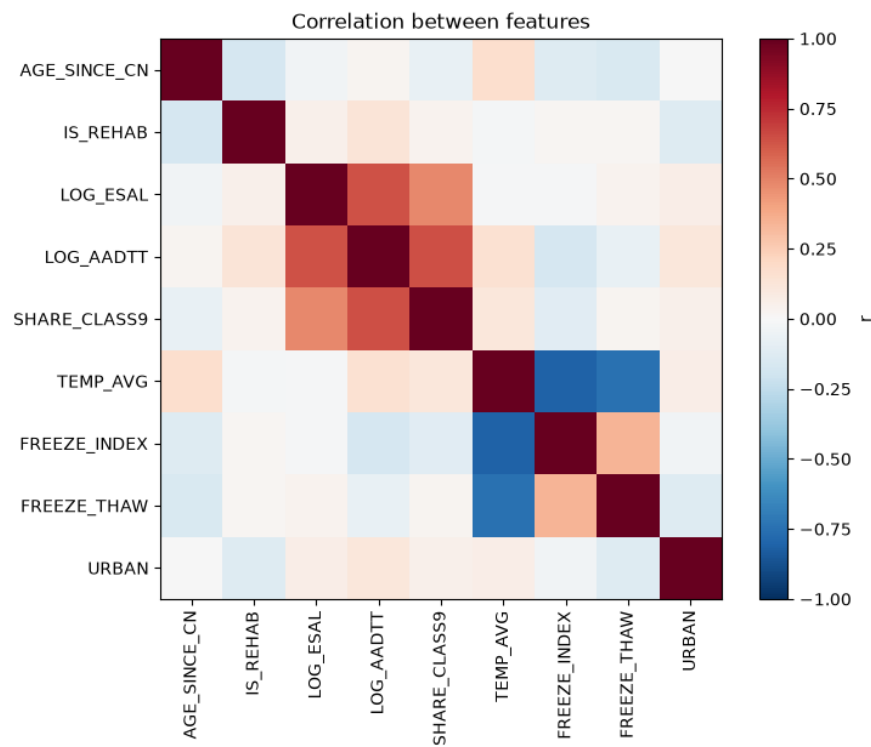
All correlations are weak (largest about 0.12 in size). Traffic is slightly negative, likely because busier roads are built thicker and maintained more (confounding).

`plt.imshow` draws the correlation table as coloured squares (a heatmap): red positive, blue negative. `xticks/yticks` put the feature names on the axes.

Code

```
corr_matrix = df[features].corr()

plt.figure(figsize=(8, 6))
plt.imshow(corr_matrix, cmap="RdBu_r", vmin=-1, vmax=1)
plt.colorbar(label="r")
plt.xticks(range(len(features)), features, rotation=90)
plt.yticks(range(len(features)), features)
plt.title("Correlation between features")
plt.show()
```



Your note in the notebook

Summary of Step 3: every feature has a weak correlation with IRI (below 0.13 in size). Temperature and freezing index overlap strongly (multicollinearity). Expect a modest R^2 from these features.

Step 4: Preparing the features

Your note in the notebook

Remove 5 impossible rows (Class 9 share above 100%), create IS_RIGID, then build X ($15,935 \times 10$) and y.

`df[condition]` keeps only rows where the condition is True (**filtering**). `.copy()` makes an independent table.

Code

```
print("Rows before:", len(df))
df = df[df["SHARE_CLASS9"] <= 1].copy()
print("Rows after: ", len(df))
```

Output

```
Rows before: 15940
Rows after:  15935
```

What the output shows

5 impossible rows removed: 15,940 to **15,935**.

Models need numbers. `(df["PAVEMENT_TYPE"] == "Rigid")` gives True/False and `.astype(int)` turns these into 1/0. A 0/1 column is a **dummy variable**.

Code

```
df["IS_RIGID"] = (df["PAVEMENT_TYPE"] == "Rigid").astype(int)
df["IS_RIGID"].value_counts()
```

Output

```
IS_RIGID
0      15113
1         822
Name: count, dtype: int64
```

What the output shows

822 rigid rows and 15,113 flexible.

The 10 features and the target. YEAR, IDs, wheel-path IRI and MRI are deliberately left out (not road properties, or the answer in disguise).

Code

```
features = ["AGE_SINCE_CN", "IS_REHAB", "IS_RIGID", "URBAN",
            "LOG_ESAL", "LOG_AADTT", "SHARE_CLASS9",
            "TEMP_AVG", "FREEZE_INDEX", "FREEZE_THAW"]
target = "IRI_AVG"
```

`.to_numpy()` turns the table into plain NumPy arrays, like in the course labs. `X.shape` is (rows, columns); `groups` holds each row's section, used only for splitting.

Code

```
X = df[features].to_numpy()
y = df[target].to_numpy()
groups = df["SECTION_ID"].to_numpy()

print("X shape:", X.shape)
print("y shape:", y.shape)
print("First row of X:", X[0])
print("First y:", y[0])
```

Output

```
X shape: (15935, 10)
y shape: (15935,)
First row of X: [ 4.          0.          0.          0.          11.24487607  5.99893656
 0.37064677 18.70000076  0.          20.          ]
First y: 1.0360000014305115
```

What the output shows

X is **(15935, 10)**: m = 15,935 examples, n = 10 features.

A compact table of each feature's range.

Code

```
df[features].describe().loc[["min", "max", "mean", "std"]].T
```

Output

	min	max	mean	std
AGE_SINCE_CN	0.000000	29.000000	4.569187	4.120681
IS_REHAB	0.000000	1.000000	0.703922	0.456540
IS_RIGID	0.000000	1.000000	0.051585	0.221194
URBAN	0.000000	1.000000	0.073423	0.260838
LOG_ESAL	0.000000	15.264397	11.692528	2.065318
LOG_AADTT	0.693147	9.627141	6.237250	1.190358
SHARE_CLASS9	0.000000	0.888889	0.470877	0.214888
TEMP_AVG	-5.400000	26.200001	11.481851	5.491661
FREEZE_INDEX	0.000000	3647.000000	383.749106	461.104034
FREEZE_THAW	0.000000	189.000000	79.926075	40.582693

What the output shows

FREEZE_INDEX reaches 3,647 while SHARE_CLASS9 stays below 1: very different scales.

Feature scaling**Your note in the notebook**

Features have very different ranges (FREEZE_INDEX up to 3,647, SHARE_CLASS9 below 1), so they are scaled to mean 0 and standard deviation 1. μ and σ are computed from the training set only, after the split.

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

Your own **z-score** function: subtract the mean μ and divide by the standard deviation σ , for all columns at once (NumPy broadcasting).

Code

```
def zscore_normalize(X, mu, sigma):  
    """Scale each column of X to mean 0 and standard deviation 1."""  
    return (X - mu) / sigma
```

Step 5: Splitting into training and test sets

Your note in the notebook

80% of roads for training, 20% for testing, split by SECTION_ID so no road appears in both sets.

`GroupShuffleSplit` shuffles **groups** (roads) rather than rows, so each road goes entirely to training or test. `test_size=0.2` puts 20% of roads in the test set; `random_state=42` fixes the shuffle so it is the same every run. `next(...)` takes the first (only) split, which returns the row numbers for each set.

Code

```
from sklearn.model_selection import GroupShuffleSplit

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(splitter.split(X, y, groups=groups))

X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]

print("Training rows:", len(train_idx))
print("Test rows:    ", len(test_idx))
```

Output

```
Training rows: 12855
Test rows:    3080
```

What the output shows

12,855 training rows and 3,080 test rows (the split is 80/20 of roads, not rows).

`set(...)` keeps each road once; `&` gives roads in both sets. This must be zero.

Code

```
train_roads = set(groups[train_idx])
test_roads = set(groups[test_idx])

print("Roads in training:", len(train_roads))
print("Roads in test:    ", len(test_roads))
print("Roads in both:    ", len(train_roads & test_roads))
```

Output

```
Roads in training: 1240
Roads in test:     310
Roads in both:     0
```

What the output shows

0 roads in both sets: the split is clean.

μ and σ are computed from the **training** data only (`axis=0` means one value per column), and both sets are scaled with them. Using test data here would leak information.

Code

```
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0)

X_train_norm = zscore_normalize(X_train, mu, sigma)
X_test_norm = zscore_normalize(X_test, mu, sigma)
```

A check that scaling worked.

Code

```
print("Training means:", X_train_norm.mean(axis=0).round(2))
print("Training stds: ", X_train_norm.std(axis=0).round(2))
print("Test means:     ", X_test_norm.mean(axis=0).round(2))
```

Output

```
Training means: [-0.  0.  0. -0. -0. -0. -0.  0. -0.  0.]
Training stds:  [1.  1.  1.  1.  1.  1.  1.  1.  1.  1.]
Test means:     [-0.02 -0.05 -0.01  0.   -0.09 -0.08 -0.06  0.    0.01 -0.01]
```

What the output shows

Training means are 0 and standard deviations 1. Test means are close to 0 but not exactly, which is correct because the test set was scaled with training statistics.

Compare one row before and after scaling.

Code

```
print("Before scaling, first training row:", X_train[0].round(2))
print("After scaling, first training row:", X_train_norm[0].round(2))
```

Output

```
Before scaling, first training row: [ 4.    0.    0.    0.   11.24  6.    0.37 18.7   0.   20.
 ]
After scaling, first training row: [-0.14 -1.56 -0.23 -0.28 -0.24 -0.22 -0.48  1.32 -0.82 -
1.48]
```

What the output shows

After scaling all values are of similar size, roughly -3 to +3.

Step 6: Linear regression from scratch

Your note in the notebook

Model $f = w \cdot x + b$; cost J = average squared error / 2; gradient descent updates w and b step by step.

$$f_{w,b}(x) = w \cdot x + b = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j} \quad b := b - \alpha \frac{\partial J}{\partial b}$$

The model $f = w \cdot x + b$. $@$ is matrix multiplication: $X @ w$ multiplies each row's features by the weights and adds them up, for every road at once.

Code

```
def predict(X, w, b):
    """Predict IRI for every row of X."""
    return X @ w + b
```

The cost J : square the errors, add them up and divide by $2m$.

Code

```
def compute_cost(X, y, w, b):
    """Mean squared error cost J(w,b), as in the course."""
    m = X.shape[0]
    errors = predict(X, w, b) - y
    cost = np.sum(errors ** 2) / (2 * m)
    return cost
```

With all weights at zero the model predicts 0 for every road, so the cost should be large.

Code

```
w_zero = np.zeros(X_train_norm.shape[1])
print("Cost with all-zero weights:", compute_cost(X_train_norm, y_train, w_zero, 0))
```

Output

```
Cost with all-zero weights: 1.0809447194260595
```

What the output shows

Cost **1.081**.

The gradient. $X.T$ is X transposed; $X.T @ errors$ gives, for every feature at once, the sum of error times feature. dj_db is the average error.

Code

```
def compute_gradient(X, y, w, b):
    """Partial derivatives of J with respect to w and b."""
    m = X.shape[0]
    errors = predict(X, w, b) - y
    dj_dw = (X.T @ errors) / m
    dj_db = np.sum(errors) / m
    return dj_dw, dj_db
```

Gradient descent: compute both gradients, update both parameters together (simultaneous update), record the cost, and print progress every 100 steps ($i \% 100 == 0$). `f"..."` strings insert values inside `{ }`.

Code

```
def gradient_descent(X, y, w_init, b_init, alpha, num_iters):
    """Run gradient descent and record the cost at every step."""
    w = w_init.copy()
    b = b_init
    J_history = []

    for i in range(num_iters):
        dj_dw, dj_db = compute_gradient(X, y, w, b)

        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        J_history.append(compute_cost(X, y, w, b))

        if i % 100 == 0:
            print(f"Iteration {i:4d}: cost = {J_history[-1]:.5f}")

    return w, b, J_history
```

Train with $\alpha = 0.1$ for 1,000 iterations.

Code

```
w_init = np.zeros(X_train_norm.shape[1])
b_init = 0.0
alpha = 0.1
iterations = 1000

w_final, b_final, J_hist = gradient_descent(X_train_norm, y_train, w_init, b_init,
                                             alpha, iterations)

print(f"\nFinal b: {b_final:.4f}")
print(f"Final cost: {J_hist[-1]:.5f}")
```

Output

```

Iteration    0: cost = 0.90756
Iteration  100: cost = 0.17009
Iteration  200: cost = 0.17007
Iteration  300: cost = 0.17007
Iteration  400: cost = 0.17007
Iteration  500: cost = 0.17007
Iteration  600: cost = 0.17007
Iteration  700: cost = 0.17007
Iteration  800: cost = 0.17007
Iteration  900: cost = 0.17007
Final b: 1.3356
Final cost: 0.17007

```

What the output shows

The cost falls from 0.908 and settles at **0.17007**. The final $b = 1.3356$ equals the average training IRI, because an average road has all scaled features at 0.

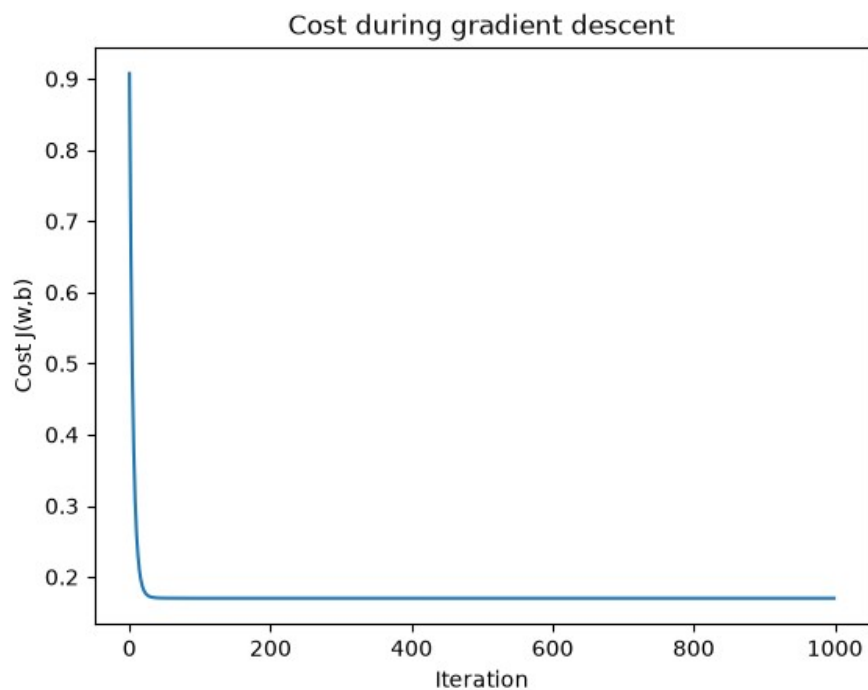
Plot the recorded costs (the learning curve).

Code

```

plt.plot(J_hist)
plt.xlabel("Iteration")
plt.ylabel("Cost J(w,b)")
plt.title("Cost during gradient descent")
plt.show()

```

**What the output shows**

A sharp drop, then flat: the model has **converged**.

Learning rate experiment

Your note in the notebook

$\alpha = 0.001$ is too slow; 0.01 to 0.5 converge; 1.0 diverges (cost becomes inf).

Train five models with different learning rates. (Only the final cost lines are shown below; the progress lines are omitted.)

Code

```
for a in [0.001, 0.01, 0.1, 0.5, 1.0]:
    w_try, b_try, J_try = gradient_descent(X_train_norm, y_train, w_init, 0.0, a, 1000)
    print(f"alpha = {a}: final cost = {J_try[-1]}")
```

Output

```
alpha = 0.001: final cost = 0.2940654903733013
alpha = 0.01: final cost = 0.1700874090169763
alpha = 0.1: final cost = 0.17007424109889763
alpha = 0.5: final cost = 0.17007424095666662
alpha = 1.0: final cost = inf
```

What the output shows

$\alpha = 0.001$ is too slow (0.294 after 1,000 steps); 0.01 to 0.5 converge to 0.1701; $\alpha = 1.0$ **diverges** (cost becomes infinite).

`pd.Series(values, index=names)` attaches feature names to the weights. With scaled features, weights can be compared directly.

Code

```
weights = pd.Series(w_final, index=features).sort_values()
weights
```

Output

```
LOG_AADTT      -0.128142
IS_REHAB       -0.013142
TEMP_AVG       -0.010310
FREEZE_THAW    -0.009423
AGE_SINCE_CN   0.009353
SHARE_CLASS9   0.018014
URBAN          0.019237
LOG_ESAL       0.047233
FREEZE_INDEX   0.067706
IS_RIGID       0.157116
dtype: float64
```

What the output shows

IS_RIGID has the largest positive weight (+0.157). LOG_AADTT (-0.128) and LOG_ESAL (+0.047) have opposite signs although both are traffic: **multicollinearity**.

scikit-learn's `LinearRegression().fit(X, y)` solves the same problem; `.coef_` is w and `.intercept_` is b .

Code

```
from sklearn.linear_model import LinearRegression

sk_model = LinearRegression()
sk_model.fit(X_train_norm, y_train)

print("Your w: ", w_final.round(4))
print("sklearn w:", sk_model.coef_.round(4))
print("Your b: ", round(b_final, 4))
print("sklearn b:", round(sk_model.intercept_, 4))
```

Output

```
Your w: [ 0.0094 -0.0131  0.1571  0.0192  0.0472 -0.1281  0.018  -0.0103  0.0677
 -0.0094]
sklearn w: [ 0.0094 -0.0131  0.1571  0.0192  0.0472 -0.1281  0.018  -0.0103  0.0677
 -0.0094]
Your b: 1.3356
sklearn b: 1.3356
```

What the output shows

Identical to 4 decimals: your gradient descent is correct.

Your note in the notebook

Note: my gradient descent weights match scikit-learn to 4 decimal places, so the code is correct. $b = 1.3356$ equals the average training IRI.

Step 7: Evaluating the model

Predictions for the training and test sets.

Code

```
y_train_pred = predict(X_train_norm, w_final, b_final)
y_test_pred = predict(X_test_norm, w_final, b_final)
```

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}^{(i)} - y^{(i)}|$$

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2}$$

$$R^2 = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2}$$

Your own R^2 : 1 minus (model's squared error) divided by (squared error of always guessing the average).

Code

```
def r_squared(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - y_true.mean()) ** 2)
    return 1 - ss_res / ss_tot

print("Train R²:", round(r_squared(y_train, y_train_pred), 3))
print("Test R²: ", round(r_squared(y_test, y_test_pred), 3))
```

Output

```
Train R²: 0.1
Test R²: 0.068
```

A **report** function giving R^2 , RMSE (big errors count more) and MAE (average error size), both in m/km.

Code

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

def report(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f"{name:10s} R² = {r2:6.3f} RMSE = {rmse:.3f} MAE = {mae:.3f}")

report("Train", y_train, y_train_pred)
report("Test", y_test, y_test_pred)
```

Output

```
Train      R² = 0.100 RMSE = 0.583 MAE = 0.420
Test       R² = 0.068 RMSE = 0.595 MAE = 0.433
```

What the output shows

Test R^2 **0.068**, RMSE **0.595 m/km**.

The baseline predicts the training average for every road (`np.full`).

Code

```
y_baseline = np.full(len(y_test), y_train.mean())
report("Baseline", y_test, y_baseline)
```

Output

```
Baseline    R² = -0.008    RMSE = 0.619    MAE = 0.450
```

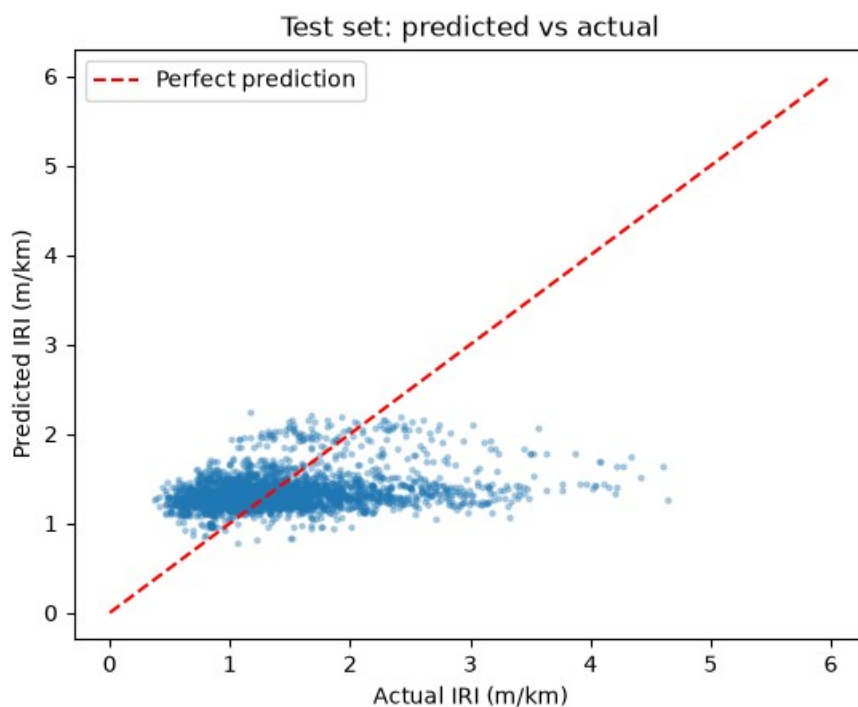
What the output shows

RMSE 0.619: the model is only about 4% better. It learned little.

Predicted against actual; the red dashed line is a perfect prediction.

Code

```
plt.scatter(y_test, y_test_pred, s=5, alpha=0.3)
plt.plot([0, 6], [0, 6], color="red", linestyle="--", label="Perfect prediction")
plt.xlabel("Actual IRI (m/km)")
plt.ylabel("Predicted IRI (m/km)")
plt.title("Test set: predicted vs actual")
plt.legend()
plt.show()
```



What the output shows

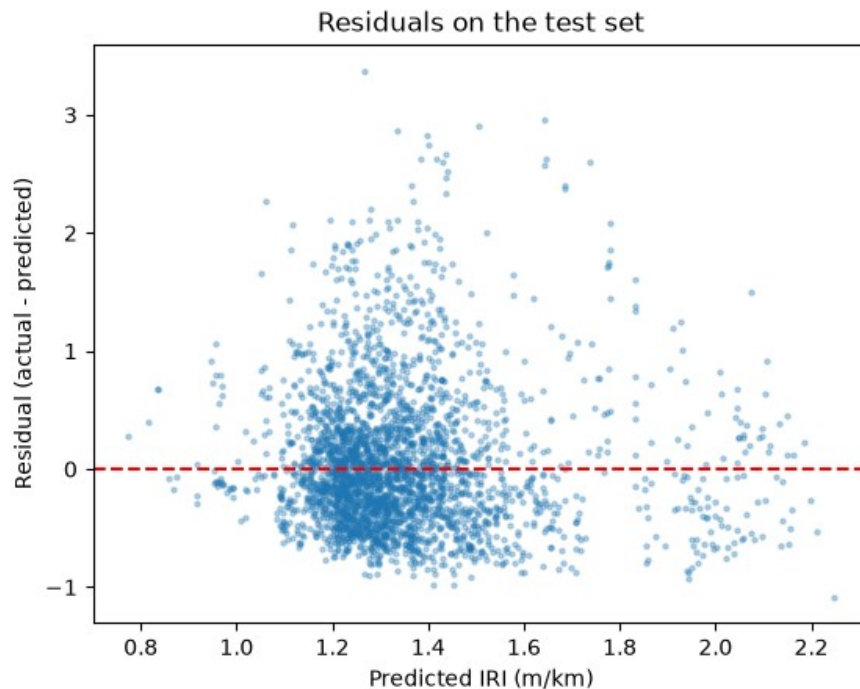
Predictions are squeezed into about 0.8 to 2.2 m/km: the model predicts "about average" for nearly every road.

A **residual** is actual minus predicted. Ideally residuals form a shapeless cloud around zero.

Code

```
residuals = y_test - y_test_pred

plt.scatter(y_test_pred, residuals, s=5, alpha=0.3)
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predicted IRI (m/km)")
plt.ylabel("Residual (actual - predicted)")
plt.title("Residuals on the test set")
plt.show()
```



What the output shows

The long upward tail shows very rough roads that the model under-predicts.

Train on $\log(\text{IRI})$ and convert predictions back with `np.exp`.

Code

```
log_model = LinearRegression()
log_model.fit(X_train_norm, np.log(y_train))

y_test_pred_log = np.exp(log_model.predict(X_test_norm))
report("Log model", y_test, y_test_pred_log)
```

Output

```
Log model    R² =  0.004    RMSE = 0.615    MAE = 0.431
```

What the output shows

Test R^2 **0.004**: worse. Plain IRI is kept as the target.

Your note in the notebook

Result: train R^2 0.100, test R^2 0.068, test RMSE 0.595 vs 0.619 for the baseline (about 4% better). Train and test are both low and close: **underfitting (high bias)**. The log target gave R^2 0.004, so plain IRI is kept.

Step 8: Polynomial features and regularisation

A **pipeline** chains steps: scale, make polynomial features (squares and products of features), scale again, fit. **StandardScaler** is scikit-learn's version of your z-score and learns μ and σ from training data only. This cell only builds and fits one pipeline; its output is the pipeline's description.

Code

```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=2, include_bias=False),
    StandardScaler(),
    LinearRegression()
)
model.fit(X_train, y_train)
```

Output

```
Pipeline(steps=[('standardscaler-1', StandardScaler()),
                 ('polynomialfeatures', PolynomialFeatures(include_bias=False)),
                 ('standardscaler-2', StandardScaler()),
                 ('linearregression', LinearRegression())])
```

The same pipeline for degrees 1 to 4. The number of features grows 10, 65, 285, 1,000.

Code

```
for degree in [1, 2, 3, 4]:
    model = make_pipeline(
        StandardScaler(),
        PolynomialFeatures(degree=degree, include_bias=False),
        StandardScaler(),
        LinearRegression()
    )
    model.fit(X_train, y_train)
    train_r2 = r2_score(y_train, model.predict(X_train))
    test_r2 = r2_score(y_test, model.predict(X_test))
    print(f"Degree {degree}:  train R² = {train_r2:.3f}    test R² = {test_r2:.3f}")
```

Output

```
Degree 1:  train R² = 0.100    test R² = 0.068
Degree 2:  train R² = 0.155    test R² = 0.073
Degree 3:  train R² = 0.244    test R² = 0.020
Degree 4:  train R² = 0.364    test R² = -1.870
```

What the output shows

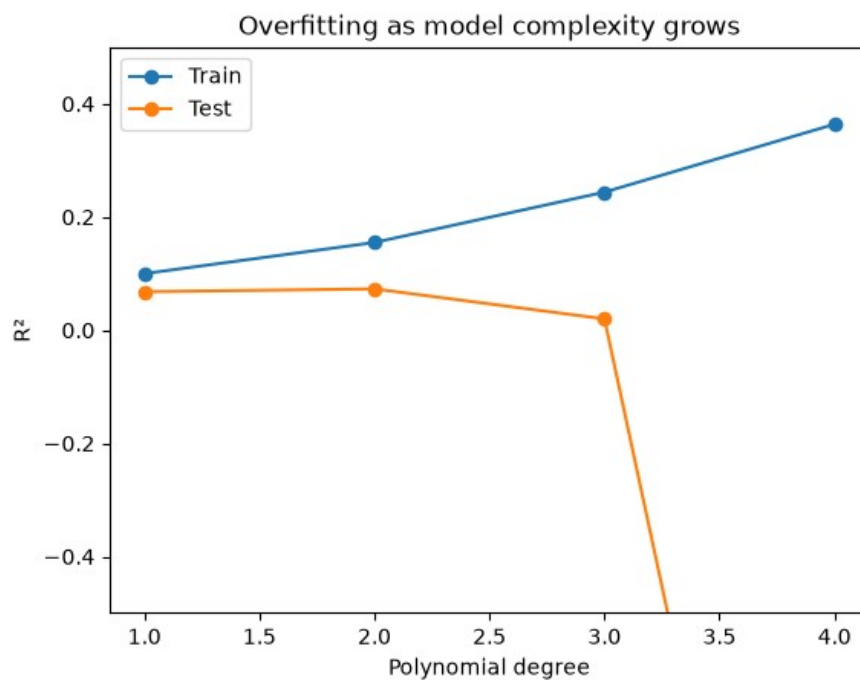
Training R^2 keeps rising (0.100 to 0.364) while test R^2 collapses to **-1.87** at degree 4: **overfitting**.

Plot the scores typed in from the previous output.

Code

```
degrees = [1, 2, 3, 4]
train_scores = [0.100, 0.155, 0.244, 0.364] # replace with your own results
test_scores = [0.068, 0.073, 0.020, -1.87]

plt.plot(degrees, train_scores, marker="o", label="Train")
plt.plot(degrees, test_scores, marker="o", label="Test")
plt.xlabel("Polynomial degree")
plt.ylabel("R²")
plt.ylim(-0.5, 0.5)
plt.legend()
plt.title("Overfitting as model complexity grows")
plt.show()
```

**Your note in the notebook**

Note: degree 4 overfits badly (train R^2 0.364, test R^2 -1.87). Regularisation (ridge) penalises large weights to control this.

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\mathbf{w}, b}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Regularised cost and gradient: the extra λ term penalises large weights. `lambda_` has an underscore because `lambda` is a reserved Python word.

Code

```
def compute_cost_reg(X, y, w, b, lambda_):
    m = X.shape[0]
    errors = predict(X, w, b) - y
    cost = np.sum(errors ** 2) / (2 * m)
    cost = cost + (lambda_ / (2 * m)) * np.sum(w ** 2)    # new line
    return cost

def compute_gradient_reg(X, y, w, b, lambda_):
    m = X.shape[0]
    errors = predict(X, w, b) - y
    dj_dw = (X.T @ errors) / m + (lambda_ / m) * w        # extra term
    dj_db = np.sum(errors) / m
    return dj_dw, dj_db
```

Cross-validation: split the training roads into 5 folds (**GroupKFold**), train on 4 and score on the 5th, five times, and average. **Ridge(alpha=lam)** is regularised regression (its **alpha** is λ , not the learning rate).

Code

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GroupKFold, cross_val_score

groups_train = groups[train_idx]

for lam in [0.01, 1, 10, 100, 1000, 10000]:
    model = make_pipeline(
        StandardScaler(),
        PolynomialFeatures(degree=3, include_bias=False),
        StandardScaler(),
        Ridge(alpha=lam)
    )
    cv_scores = cross_val_score(model, X_train, y_train, groups=groups_train,
                                cv=GroupKFold(n_splits=5), scoring="r2")
    print(f"lambda = {lam:>6}: cross-validated R2 = {cv_scores.mean():.3f}")
```

Output

```
lambda = 0.01: cross-validated R2 = -0.016
lambda = 1: cross-validated R2 = -0.002
lambda = 10: cross-validated R2 = 0.040
lambda = 100: cross-validated R2 = 0.097
lambda = 1000: cross-validated R2 = 0.140
lambda = 10000: cross-validated R2 = 0.124
```

What the output shows

Best $\lambda = 1000$ (R^2 0.140). Too little λ overfits, too much underfits.

The chosen model, trained on all training data and tested once.

Code

```
best_model = make_pipeline(
    StandardScaler(),
    PolynomialFeatures(degree=3, include_bias=False),
    StandardScaler(),
    Ridge(alpha=1000)
)
best_model.fit(X_train, y_train)
report("Train", y_train, best_model.predict(X_train))
report("Test", y_test, best_model.predict(X_test))
```

Output

Train	$R^2 = 0.214$	RMSE = 0.545	MAE = 0.388
Test	$R^2 = 0.069$	RMSE = 0.595	MAE = 0.432

What the output shows

Test R^2 **0.069**, no better than the straight line: the features lack information.

Your note in the notebook

Result: cross-validation chose $\lambda = 1000$ (R^2 0.140), but test R^2 is still 0.069. More flexibility does not help: the features lack information.

Step 9: Adding previous IRI

Your note in the notebook

PREV_IRI = IRI at the previous survey of the same road and construction event; GAP_YRS = years since that survey.

Sort, group by road and construction event, and `.shift(1)` to get each survey's previous IRI. The first survey of each group has none (NaN).

Code

```
df = df.sort_values(["SECTION_ID", "CONSTRUCTION_NO", "YEAR"])

grouped = df.groupby(["SECTION_ID", "CONSTRUCTION_NO"])
df["PREV_IRI"] = grouped["IRI_AVG"].shift(1)
```

Check on section 01_0502.

Code

```
df[df["SECTION_ID"] == "01_0502"][["YEAR", "CONSTRUCTION_NO", "IRI_AVG", "PREV_IRI"]]
```

Output

	YEAR	CONSTRUCTION_NO	IRI_AVG	PREV_IRI
0	1991	1	1.0360	NaN
1	1992	2	0.7745	NaN
2	1994	2	0.8540	0.7745
3	1996	2	0.8193	0.8540
4	1999	2	0.8143	0.8193
5	2001	2	0.8377	0.8143
6	2002	2	0.8422	0.8377
7	2003	2	0.8877	0.8422
8	2004	2	0.9545	0.8877
9	2005	2	1.0105	0.9545
10	2006	2	1.0141	1.0105
11	2008	2	1.1096	1.0141
12	2009	2	1.0689	1.1096
13	2013	2	1.2232	1.0689
14	2014	2	1.3601	1.2232
15	2015	2	1.2777	1.3601
16	2017	2	1.2893	1.2777

What the output shows

Each row's PREV_IRI is the IRI of the row above it within the same construction event.

Years since the previous survey, using the same `shift` trick on YEAR.

Code

```
df["GAP_YRS"] = df["YEAR"] - grouped["YEAR"].shift(1)
df["GAP_YRS"].describe()
```

Output

```
count    11581.000000
mean      1.471289
std       0.836851
min       1.000000
25%       1.000000
50%       1.000000
75%       2.000000
max       9.000000
Name: GAP_YRS, dtype: float64
```

What the output shows

Most gaps are 1 or 2 years (max 9).

`dropna(subset=["PREV_IRI"])` removes first surveys; the result is `df_lag`.

Code

```
print("Rows before:", len(df))
df_lag = df.dropna(subset=["PREV_IRI"]).copy()
print("Rows after: ", len(df_lag))
```

Output

```
Rows before: 15935
Rows after:  11581
```

What the output shows

15,935 to **11,581** rows.

Correlation between IRI and previous IRI.

Code

```
df_lag[["IRI_AVG", "PREV_IRI"]].corr()
```

Output

```
      IRI_AVG  PREV_IRI
IRI_AVG  1.000000  0.961115
PREV_IRI  0.961115  1.000000
```

What the output shows

$r = 0.96$, far stronger than any feature in Step 3.

Two models on the same rows: A (10 features) and B (plus PREV_IRI and GAP_YRS). The loop unpacks each (name, feature list) pair.

Code

```

features_lag = features + ["PREV_IRI", "GAP_YRS"]

y_lag = df_lag["IRI_AVG"].to_numpy()
groups_lag = df_lag["SECTION_ID"].to_numpy()

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
tr, te = next(splitter.split(df_lag, y_lag, groups=groups_lag))

for name, cols in [("A: without previous IRI", features),
                   ("B: with previous IRI", features_lag)]:
    X_lag = df_lag[cols].to_numpy()
    model = make_pipeline(StandardScaler(), LinearRegression())
    model.fit(X_lag[tr], y_lag[tr])
    print(name)
    report(" Train", y_lag[tr], model.predict(X_lag[tr]))
    report(" Test", y_lag[te], model.predict(X_lag[te]))

```

Output

```

A: without previous IRI
Train    R² =  0.091   RMSE = 0.562   MAE = 0.404
Test     R² =  0.089   RMSE = 0.583   MAE = 0.412
B: with previous IRI
Train    R² =  0.931   RMSE = 0.155   MAE = 0.090
Test     R² =  0.914   RMSE = 0.179   MAE = 0.099

```

What the output shows

Test R^2 jumps from **0.089** to **0.914**.

The persistence baseline: next IRI = previous IRI, no learning.

Code

```

persistence = df_lag["PREV_IRI"].to_numpy()[te]
report("Persistence", y_lag[te], persistence)

```

Output

```

Persistence    R² =  0.897   RMSE = 0.197   MAE = 0.106

```

What the output shows

Persistence already scores **0.897** (RMSE 0.197). The model adds 0.897 to 0.914, about 9% lower error.

Standardised coefficients of model B, sorted by size.

Code

```

model_B = make_pipeline(StandardScaler(), LinearRegression())
model_B.fit(df_lag[features_lag].to_numpy()[tr], y_lag[tr])

coefs = pd.Series(model_B[-1].coef_, index=features_lag)
coefs.sort_values(key=abs, ascending=False)

```

Output

```
PREV_IRI      0.570442
GAP_YRS       0.029723
FREEZE_INDEX  0.016554
SHARE_CLASS9  0.011217
IS_RIGID      -0.009511
URBAN         0.008734
TEMP_AVG      0.006509
IS_REHAB      -0.003881
AGE_SINCE_CN  -0.002063
FREEZE_THAW   0.001864
LOG_AADTT     -0.001268
LOG_ESAL      0.001238
dtype: float64
```

What the output shows

PREV_IRI dominates (0.570), then GAP_YRS (0.030), FREEZE_INDEX (0.017) and SHARE_CLASS9 (0.011).

Predict the **change** in IRI since the last survey.

Code

```
change = y_lag - df_lag["PREV_IRI"].to_numpy()

X_B = df_lag[features_lag].to_numpy()
change_model = make_pipeline(StandardScaler(), LinearRegression())
change_model.fit(X_B[tr], change[tr])

print("R² for predicting the change:", round(r2_score(change[te],
change_model.predict(X_B[te])), 3))
```

Output

```
R² for predicting the change: 0.056
```

What the output shows

Only **5.6%** of the change is explained: predicting deterioration remains hard.

Your note in the notebook

Result: with previous IRI, test R^2 0.914 (RMSE 0.179), but persistence (next = previous) already gives 0.897 (RMSE 0.197). The other features cut error by about 9% and explain only about 6% of the change in IRI. After PREV_IRI, the most useful features are GAP_YRS, FREEZE_INDEX and SHARE_CLASS9.

Decision trees ask yes/no questions; `max_depth` limits how many in a row (`None` = no limit).

Code

```
from sklearn.tree import DecisionTreeRegressor

X_A = df_lag[features].to_numpy()

for depth in [2, 4, 6, 10, None]:
    tree = DecisionTreeRegressor(max_depth=depth, random_state=42)
    tree.fit(X_A[tr], y_lag[tr])
    train_r2 = r2_score(y_lag[tr], tree.predict(X_A[tr]))
    test_r2 = r2_score(y_lag[te], tree.predict(X_A[te]))
    print(f"max_depth = {str(depth):>4}:  train R² = {train_r2:.3f}    test R² = {test_r2:.3f}")
```

Output

```
max_depth = 2:  train R² = 0.077    test R² = 0.083
max_depth = 4:  train R² = 0.133    test R² = 0.112
max_depth = 6:  train R² = 0.228    test R² = 0.158
max_depth = 10: train R² = 0.465    test R² = 0.065
max_depth = None: train R² = 0.965    test R² = -0.170
```

What the output shows

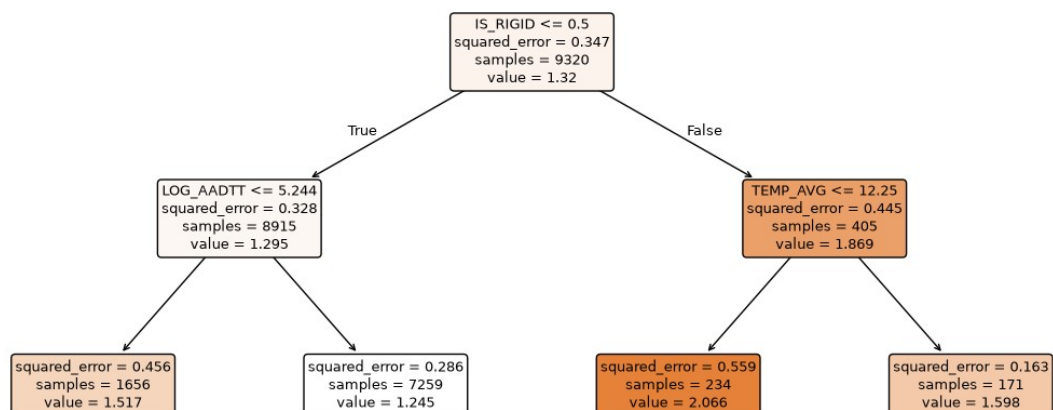
Unlimited depth: train 0.965 but test **-0.170**. Depth is the complexity dial.

Draw a small depth-2 tree. Each box shows the question, error, number of rows and predicted IRI.

Code

```
from sklearn.tree import plot_tree

small_tree = DecisionTreeRegressor(max_depth=2).fit(X_A[tr], y_lag[tr])
plt.figure(figsize=(14, 6))
plot_tree(small_tree, feature_names=features, filled=True, rounded=True, fontsize=9)
plt.show()
```



A random forest averages 300 trees, each trained on random samples; `min_samples_leaf=5` stops memorising single rows; `n_jobs=-1` uses all processor cores.

Code

```
from sklearn.ensemble import RandomForestRegressor

forest_A = RandomForestRegressor(n_estimators=300, min_samples_leaf=5,
                                n_jobs=-1, random_state=42)
forest_A.fit(X_A[tr], y_lag[tr])
report("Forest A train", y_lag[tr], forest_A.predict(X_A[tr]))
report("Forest A test", y_lag[te], forest_A.predict(X_A[te]))
```

Output

```
Forest A train  R² =  0.712   RMSE = 0.316   MAE = 0.210
Forest A test   R² =  0.351   RMSE = 0.492   MAE = 0.332
```

What the output shows

Test R^2 **0.351**, suspiciously high for the same weak features.

Checking for site leakage**Your note in the notebook**

Forest A scored test R^2 0.351, suspiciously high. SPS sections at the same site share traffic and climate, so sister sections may be on both sides of the split.

SPS sections at one site share traffic and climate. `np.where` builds a site ID: state code plus the first two characters of SHRP_ID for SPS rows, the section itself for GPS rows.

Code

```
df_lag["SITE_ID"] = np.where(
    df_lag["GPS_SPS"] == "S",
    df_lag["STATE_CODE"].astype(str).str.zfill(2) + "_" + df_lag["SHRP_ID"].str[:2],
    df_lag["SECTION_ID"]
)

print("Sections:", df_lag["SECTION_ID"].nunique())
print("Sites:   ", df_lag["SITE_ID"].nunique())
```

Output

```
Sections: 1527
Sites:    792
```

What the output shows

1,527 sections but only **792** sites.

Five-fold cross-validation grouped by section and then by site, for four models.

Code

```

from sklearn.model_selection import GroupKFold, cross_val_score

X_B = df_lag[features_lag].to_numpy()
sites = df_lag["SITE_ID"].to_numpy()
sections = df_lag["SECTION_ID"].to_numpy()
cv = GroupKFold(n_splits=5)

for split_name, grp in [("by SECTION", sections), ("by SITE", sites)]:
    print(f"\nSplit {split_name}")
    for model_name, model, X_use in [
        ("Linear A", LinearRegression(), X_A),
        ("Forest A", RandomForestRegressor(n_estimators=200, min_samples_leaf=5,
n_jobs=-1, random_state=42), X_A),
        ("Linear B", LinearRegression(), X_B),
        ("Forest B", RandomForestRegressor(n_estimators=200, min_samples_leaf=5,
n_jobs=-1, random_state=42), X_B),
    ]:
        scores = cross_val_score(model, X_use, y_lag, groups=grp, cv=cv, scoring="r2")
        print(f" {model_name}: mean R2 = {scores.mean():.3f}")

```

Output

```

Split by SECTION
Linear A: mean R2 = 0.082
Forest A: mean R2 = 0.345
Linear B: mean R2 = 0.927
Forest B: mean R2 = 0.927
Split by SITE
Linear A: mean R2 = 0.027
Forest A: mean R2 = 0.092
Linear B: mean R2 = 0.924
Forest B: mean R2 = 0.920

```

What the output shows

Forest A drops from **0.345** (by section) to **0.092** (by site): most of its skill came from recognising sites. Models with previous IRI barely change.

Split by site and measure **permutation importance**: shuffle one feature and see how much R^2 drops.

Code

```

from sklearn.model_selection import GroupShuffleSplit
from sklearn.inspection import permutation_importance

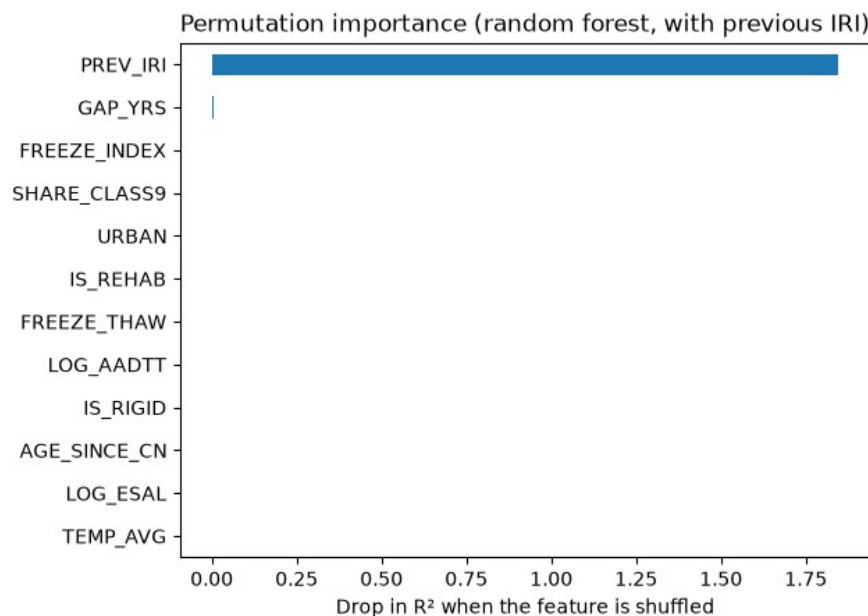
site_split = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
tr_s, te_s = next(site_split.split(X_B, y_lag, groups=sites))

forest_B = RandomForestRegressor(n_estimators=300, min_samples_leaf=5, n_jobs=-1,
random_state=42)
forest_B.fit(X_B[tr_s], y_lag[tr_s])

perm = permutation_importance(forest_B, X_B[te_s], y_lag[te_s], n_repeats=5,
random_state=42)
importance = pd.Series(perm.importances_mean, index=features_lag).sort_values()

importance.plot(kind="barh")
plt.xlabel("Drop in R² when the feature is shuffled")
plt.title("Permutation importance (random forest, with previous IRI)")
plt.show()

```



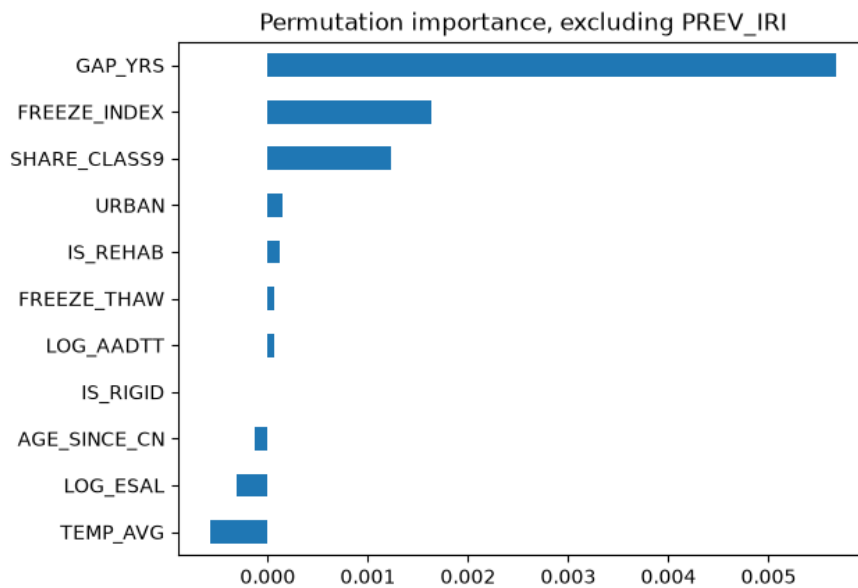
The same chart without PREV_IRI, so the small features are visible.

Code

```

importance.drop("PREV_IRI").plot(kind="barh")
plt.title("Permutation importance, excluding PREV_IRI")
plt.show()

```



What the output shows

After PREV_IRI, GAP_YRS, FREEZE_INDEX and SHARE_CLASS9 matter most.

Your note in the notebook

Result: splitting by site drops Forest A from 0.345 to 0.092, so most of its skill was recognising sites. With previous IRI, forest and linear models perform the same (about 0.92). PREV_IRI dominates importance; GAP_YRS is next.

The classification target: 1 if IRI above 2.68 m/km (poor), else 0.

Code

```
y_poor = (df_lag["IRI_AVG"] > 2.68).astype(int).to_numpy()

print("Share of poor measurements:", y_poor.mean().round(3))
print("Poor roads in the test set:", y_poor[te_s].sum(), "of", len(te_s))
```

Output

```
Share of poor measurements: 0.04
Poor roads in the test set: 97 of 2295
```

What the output shows

Only 4% are poor; 97 of 2,295 in the test set.

Scale with the training statistics of the site split.

Code

```
mu_B = X_B[tr_s].mean(axis=0)
sigma_B = X_B[tr_s].std(axis=0)
Xtr_c = zscore_normalize(X_B[tr_s], mu_B, sigma_B)
Xte_c = zscore_normalize(X_B[te_s], mu_B, sigma_B)
ytr_c, yte_c = y_poor[tr_s], y_poor[te_s]
```

$$g(z) = \frac{1}{1 + e^{-z}} \quad z = \mathbf{w} \cdot \mathbf{x} + b$$

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log f^{(i)} + (1 - y^{(i)}) \log (1 - f^{(i)})]$$

Logistic regression pieces: the **sigmoid** squashes any number into a probability between 0 and 1; the **log loss** cost punishes confident wrong answers; **np.clip** avoids log(0). The gradient has the same form as linear regression.

Code

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_cost_logistic(X, y, w, b):
    m = X.shape[0]
    f = sigmoid(X @ w + b)
    f = np.clip(f, 1e-15, 1 - 1e-15)
    cost = -np.sum(y * np.log(f) + (1 - y) * np.log(1 - f)) / m
    return cost

def compute_gradient_logistic(X, y, w, b):
    m = X.shape[0]
    errors = sigmoid(X @ w + b) - y
    dj_dw = (X.T @ errors) / m
    dj_db = np.sum(errors) / m
    return dj_dw, dj_db
```

Gradient descent for logistic regression.

Code

```
w = np.zeros(Xtr_c.shape[1])
b = 0.0
alpha = 0.5

print("Starting cost:", round(compute_cost_logistic(Xtr_c, ytr_c, w, b), 4))

for i in range(5000):
    dj_dw, dj_db = compute_gradient_logistic(Xtr_c, ytr_c, w, b)
    w = w - alpha * dj_dw
    b = b - alpha * dj_db
    if i % 1000 == 0:
        print(f"Iteration {i}: cost = {compute_cost_logistic(Xtr_c, ytr_c, w, b):.4f}")

w_log, b_log = w, b
```

Output

```
Starting cost: 0.6931
Iteration 0: cost = 0.5865
Iteration 1000: cost = 0.0407
Iteration 2000: cost = 0.0391
Iteration 3000: cost = 0.0387
Iteration 4000: cost = 0.0385
```

What the output shows

Starting cost **0.6931** = $\ln 2$, the cost of a 50/50 guess; it falls to about 0.038.

Turn probabilities into yes/no at 0.5.

Code

```
prob_poor = sigmoid(Xte_c @ w_log + b_log)
pred_poor = (prob_poor >= 0.5).astype(int)
print("Predicted poor:", pred_poor.sum(), " Actually poor:", yte_c.sum())
```

Output

```
Predicted poor: 72   Actually poor: 97
```

What the output shows

72 predicted poor against 97 actually poor.

scikit-learn's version. `predict_proba(...)[:, 1]` gives the probability of poor.

Code

```
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
clf.fit(X_B[tr_s], y_poor[tr_s])

prob_sk = clf.predict_proba(X_B[te_s])[ :, 1]
pred_sk = clf.predict(X_B[te_s])

print("Your w:   ", w_log.round(2))
print("sklearn w:", clf[-1].coef_[0].round(2))
```

Output

```
Your w:   [-0.16 -0.05 -0.14 -0.22 -0.4   0.51 -0.09 -0.04  0.14  0.16  3.28  0.59]
sklearn w: [-0.16 -0.06 -0.13 -0.22 -0.4   0.51 -0.09 -0.04  0.15  0.17  3.33  0.61]
```

What the output shows

Weights are close to yours (scikit-learn adds light regularisation by default).

Evaluating the classifier**Your note in the notebook**

Precision = share of flagged roads that are really poor. Recall = share of poor roads that were caught.

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A reporting function built on the **confusion matrix**. Precision = $TP/(TP+FP)$; Recall = $TP/(TP+FN)$.

`.ravel()` flattens the 2x2 matrix into TN, FP, FN, TP.

Code

```
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

def class_report(name, y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    print(f"{name:22s} acc={accuracy_score(y_true, y_pred):.3f} "
          f"prec={precision_score(y_true, y_pred, zero_division=0):.3f} "
          f"rec={recall_score(y_true, y_pred):.3f} "
          f"f1={f1_score(y_true, y_pred):.3f} TP={tp} FN={fn} FP={fp}")
```

Compare with two baselines.

Code

```
yt = y_poor[te_s]
always_good = np.zeros(len(yt), dtype=int)
persistence = (df_lag["PREV_IRI"].to_numpy()[te_s] > 2.68).astype(int)

class_report("Always 'not poor'", yt, always_good)
class_report("Persistence", yt, persistence)
class_report("Logistic (0.5)", yt, pred_sk)
```

Output

Always 'not poor'	acc=0.958	prec=0.000	rec=0.000	f1=0.000	TP=0 FN=97 FP=0
Persistence	acc=0.981	prec=0.922	rec=0.608	f1=0.733	TP=59 FN=38 FP=5
Logistic (0.5)	acc=0.981	prec=0.875	rec=0.649	f1=0.746	TP=63 FN=34 FP=9

What the output shows

Always "not poor" is 95.8% accurate yet catches no poor roads. Logistic regression catches 63 of 97 (recall 0.649) at precision 0.875.

Try lower thresholds.

Code

```
for threshold in [0.5, 0.3, 0.1]:
    pred_t = (prob_sk >= threshold).astype(int)
    class_report(f"Threshold {threshold}", yt, pred_t)
```

Output

Threshold 0.5	acc=0.981	prec=0.875	rec=0.649	f1=0.746	TP=63 FN=34 FP=9
Threshold 0.3	acc=0.981	prec=0.765	rec=0.804	f1=0.784	TP=78 FN=19 FP=24
Threshold 0.1	acc=0.959	prec=0.512	rec=0.876	f1=0.646	TP=85 FN=12 FP=81

What the output shows

Lower threshold: more poor roads caught, more false alarms (0.3 catches 78; 0.1 catches 85 with 81 false alarms).

`class_weight="balanced"` makes each poor road count about 24 times more.

Code

```
clf_bal = make_pipeline(StandardScaler(),
                        LogisticRegression(class_weight="balanced", max_iter=1000))
clf_bal.fit(X_B[tr_s], y_poor[tr_s])
class_report("Balanced weights", yt, clf_bal.predict(X_B[te_s]))
```

Output

```
Balanced weights      acc=0.932  prec=0.377  rec=0.918  f1=0.535  TP=89 FN=8 FP=147
```

What the output shows

Recall 0.918 (only 8 missed) but precision 0.377.

The same classifier without previous IRI.

Code

```
X_A_all = df_lag[features].to_numpy()
clf_A = make_pipeline(StandardScaler(),
                     LogisticRegression(class_weight="balanced", max_iter=1000))
clf_A.fit(X_A_all[tr_s], y_poor[tr_s])
class_report("No previous IRI", yt, clf_A.predict(X_A_all[te_s]))
```

Output

```
No previous IRI      acc=0.696  prec=0.065  rec=0.464  f1=0.114  TP=45 FN=52 FP=645
```

What the output shows

Only 6.5% of flagged roads are really poor: traffic and climate alone cannot find poor roads.

Logistic weights sorted by size (positive raises the probability of poor).

Code

```
pd.Series(clf[-1].coef_[0], index=features_lag).sort_values(key=abs, ascending=False)
```

Output

```
PREV_IRI      3.330429
GAP_YRS       0.605659
LOG_AADTT      0.509053
LOG_ESAL      -0.402961
URBAN         -0.220142
FREEZE_THAW   0.167545
AGE_SINCE_CN  -0.164633
FREEZE_INDEX  0.146130
IS_RIGID      -0.133266
SHARE_CLASS9  -0.093601
IS_REHAB      -0.055801
TEMP_AVG      -0.039695
dtype: float64
```

What the output shows

PREV_IRI (+3.3) and GAP_YRS (+0.6) lead; the two traffic measures again have opposite signs.

Conclusions

Your note in the notebook

Result: always "not poor" is 95.8% accurate but catches no poor roads. Logistic regression (threshold 0.5) catches 65% of poor roads at 88% precision, vs 61% and 92% for persistence. Lowering the threshold to 0.3 catches 80%. Without previous IRI, only 6.5% of flags are correct.

1. Traffic, climate and section attributes alone explain little roughness (R^2 below 0.1).
2. Previous IRI dominates; the other features add about 9% improvement over persistence.
3. How the data is split (by section or by site) changes the conclusions for flexible models.
4. Always compare with simple baselines.

Summary of results

Regression results from the notebook (split by section)

Stage	Test R ²	Test RMSE (m/km)
Guess the average	0.00	0.619
Linear, traffic and climate (Step 7)	0.068	0.595
Polynomial degree 3 + ridge (Step 8)	0.069	0.595
Persistence: previous IRI (Step 9)	0.897	0.197
Linear with previous IRI (Step 9)	0.914	0.179

Five-fold cross-validated R² (Step 10)

Model	Split by section	Split by site
Linear A (no previous IRI)	0.082	0.027
Forest A (no previous IRI)	0.345	0.092
Linear B (with previous IRI)	0.927	0.924
Forest B (with previous IRI)	0.927	0.920

Classification of poor condition (Step 11, split by site)

Classifier	Precision	Recall	Poor missed (of 97)
Always "not poor"	0.000	0.000	97
Persistence	0.922	0.608	38
Logistic, threshold 0.5	0.875	0.649	34
Logistic, threshold 0.3	0.765	0.804	19
Logistic, balanced weights	0.377	0.918	8
Without previous IRI	0.065	0.464	52

Four conclusions

- Traffic, climate and section attributes on their own explain very little roughness.
- Previous condition dominates; time between surveys, frost and heavy trucks add a little.
- How the data is split changes the conclusions for flexible models such as random forests.
- Always compare against simple baselines: the average, persistence, and "always not poor".

Note on the Project Report

The Project Report uses an even stricter grouping (sections sharing the same climate record), under which the forest without previous IRI falls to about zero. The conclusions are the same, only stronger.